AZ-900

AZ-900 Microsoft Azure Fundamentals

Edition 2 · Skills measured as of 2026-07-20

Source: https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/az-900

Built 2026-10-13. An independent study aid, not Microsoft material.

Contents

AZ-900: Microsoft Azure Fundamentals 3

Scope and weights 3

How to use this book 3

Edition 3

1. Cloud concepts (25-30%) 4

1.1 What cloud computing is 4

1.2 Why organizations use the cloud 7

1.3 Cloud service types: IaaS, PaaS, SaaS 10

2. Azure architecture and services (35-40%) 13

2.1 Azure's physical and organizational structure 13

2.2 Compute and networking 17

2.3 Storage and data movement 21

2.4 Identity, access, and security 27

3. Azure management and governance (30-35%) 33

3.1 Costs in Azure 33

3.2 Governance and compliance 37

3.3 Managing and deploying resources 41

3.4 Monitoring and advice 45

AZ-900: Microsoft Azure Fundamentals

An original study book for the AZ-900 exam. Content is written in the author's own words from official Microsoft Learn documentation and organized around the intent of the Microsoft AZ-900 study guide (skills measured as of July 20, 2026). It is an independent study aid, not Microsoft material.

Scope and weights

Chapter	Exam weight
1. Cloud concepts	25–30%
2. Azure architecture and services	35–40%
3. Azure management and governance	30–35%

Source record and extraction notes: CURRICULUM.md. Outcome-to-section-to-question map: coverage.md. The map supports review; it is not a guarantee of exhaustive coverage.

How to use this book

Read each topic in order or jump to a gap via the coverage map. Every topic follows the same pattern: learning intent, core concepts, distinctions and limits, a worked scenario, hands-on steps where relevant, exam application, and self-check questions. Then take the matching 100-question mock exam (`mock_exams/az_900/`) and revisit weak topics, using the `objective:` tags to find them.

Azure changes often. Items marked **Verify** are version-sensitive; check the linked Microsoft documentation before relying on them. The guide scopes the exam; it is not a complete course, and enrichment beyond it is noted in the text.

Edition

Edition 2 (2026-10-11), written against the July 20, 2026 skills outline. Content is drafted and not yet human-reviewed.

CHAPTER 1

Cloud concepts (25-30%)

This chapter introduces cloud computing as a way to access technology capabilities over the internet without owning and operating every part of the underlying infrastructure. It explains how deployment and service models change control, responsibility, and cost, then connects those choices to availability, adaptability, security, and day-to-day operations. The goal is to recognize what the cloud can provide—and what an organization must still design, configure, and manage.

1.1 What cloud computing is

Learning intent

Be able to explain cloud computing in plain language, distinguish public, private, and hybrid deployment models, and select a model for a stated need. Understand the shared responsibility model, consumption-based billing, common pricing approaches, and serverless computing as a cloud execution approach.

Core concepts

Cloud computing (`cc-define-cloud`). Cloud computing delivers services such as compute, storage, databases, and networking over the internet. Instead of first buying and installing physical infrastructure, an organization can provision resources from a cloud provider. The defining idea is access to computing capability, not whether an application has a web interface.

Shared responsibility (`cc-shared-responsibility`). A cloud provider and customer divide operational and security duties. The provider operates the physical datacenter and supporting infrastructure; the customer remains accountable for its data, identities, and the parts of the service it controls. The division varies by service type: a virtual machine leaves the customer more to manage than a hosted application.

The table is a broad orientation, not a substitute for a service's terms or detailed control guidance. “Shared” indicates that both parties have related responsibilities; it does not mean every task is split evenly.

Responsibility area	On-premises	IaaS	PaaS	SaaS
Customer data, configurations, identities and users	Customer	Customer	Customer	Customer
Client devices	Customer	Customer	Customer	Shared
Applications	Customer	Customer	Shared	Shared
Network controls	Customer	Customer	Shared	Provider
Operating system	Customer	Customer	Provider	Provider
Physical hosts, physical network, datacenter	Customer	Provider	Provider	Provider

Cloud models (`cc-cloud-models`). A *deployment model* describes how cloud resources are organized and who can use them; it is different from a service model such as IaaS or SaaS.

Deployment model	What it means	Common trade-off
Public cloud	A third-party provider offers cloud resources for use by customers.	Rapid provisioning and broad provider services, with less direct control of the underlying infrastructure.
Private cloud	A cloud environment is dedicated to one organization. It can be hosted on-site or by a third party.	More control over the environment, while the organization retains more responsibility for infrastructure and operations.
Hybrid cloud	Public and private cloud environments are connected and used together.	Greater flexibility in deciding where workloads and data run, with additional integration and management work.

Cloud model use cases (`cc-cloud-model-use-cases`). Choose from requirements, not from a claim that one model is universally best. Public cloud can suit a team that needs to provision capacity quickly or avoid purchasing its own datacenter hardware. A private cloud can fit an organization that needs a dedicated environment and is prepared to retain greater infrastructure control and responsibility. A hybrid model can be useful when some workloads need to remain in a private environment while others benefit from public-cloud scale, or when public resources temporarily supplement private capacity. Security, legal obligations, skills, costs, and existing systems all affect the decision.

Consumption model (`cc-consumption-model`). Under a consumption-based approach, a customer pays for measured services it uses rather than purchasing all capacity in advance. Resources can often be added as needs grow and removed when they are no longer needed. This can reduce idle capacity and upfront infrastructure purchases, but "pay for what you use" does not mean cloud services are automatically inexpensive: unused resources, unexpectedly high demand and poor configuration can still create costs. Monitoring and deliberate capacity choices matter.

Pricing models (`cc-pricing-models`). In the AZ-900 comparison, purchasing and maintaining physical infrastructure is commonly presented as capital expenditure (CapEx), while metered cloud services are commonly presented as operational expenditure (OpEx). This is a budgeting distinction, not a promise that every cloud charge is variable. *Pay-as-you-go* charges follow service consumption. *Reservations* commit to qualifying usage for a defined term in return for a discount on matching usage; they can suit stable demand but may not fit uncertain or mismatched use. Terms, eligible services, and payment choices vary.

> **Verify:** Confirm current rates, eligible products, reservation terms, and billing choices in the Azure pricing calculator and current product pricing information before making a purchasing decision. An estimate is not a guaranteed invoice.

Serverless (`cc-serverless`). Serverless is a cloud execution approach in which the provider manages the servers and much of the infrastructure operation, while the customer concentrates on application logic. It does not mean that servers do not exist. Azure Functions, for example, supports event-driven code such as

responding to a file upload, a scheduled event, or a message. Hosting options and their scaling and billing behavior differ, so do not assume every serverless workload scales identically or is billed only when code runs.

How it works, distinctions, limits, and defaults

Keep three labels separate: the **deployment model** (public/private/hybrid) describes the environment; the **service model** (IaaS/PaaS/SaaS) describes the stack the provider operates; the **billing model** describes how charges are calculated. A public-cloud customer might use a PaaS database and pay according to measured usage—one label does not determine the others.

Moving a workload to the cloud does not hand off all security obligations. Customers still protect data, manage access, configure services, and monitor what they control. Provider-managed infrastructure does not make a workload secure by default; the boundary depends on the service and its configuration.

> **Conflict:** Microsoft's AZ-900 cloud-compute learning unit (dated 2024) describes identity and access as shared for PaaS and SaaS, explaining that the customer manages users, roles, and policies while the provider operates the authentication platform. Microsoft's Shared Responsibility article (dated 2026) lists “identities and users” and access management among customer responsibilities across cloud types. These descriptions use different levels of detail and their broad wording does not align exactly. A useful operational distinction is between operating an identity service and configuring the organization's users, roles, and access controls; verify the responsibility for the specific service and feature rather than treating “shared” as a blanket answer.

Example or worked scenario

A small retailer is preparing a one-week online promotion. It needs additional capacity during the campaign, does not want to purchase servers that may sit idle afterward, and already has an internal policy requiring sensitive finance records to stay in its private environment.

The retailer could host the campaign site in public cloud, scale it back afterward, and retain the finance workload privately in a hybrid design. PaaS suits developers who want to avoid OS patching; a legacy application requiring OS-level changes may call for IaaS. Consumption-based charges can follow campaign usage, while reservations are better evaluated for steady eligible demand than a short uncertain peak. In each case, the retailer manages its data, identities, and applicable controls.

Exam or practical application

For a scenario, identify the key requirement: OS control points to IaaS, infrastructure upkeep in a dedicated environment to private cloud, and connected private/public resources to hybrid. For costs, distinguish owned infrastructure from metered services and reservations from pay-as-you-go. For security, identify the service model and specific control.

Self-check questions

1. A team wants to keep full control of a server operating system while avoiding ownership of physical hosts. Which arrangement best fits?

   Answer: IaaS in a cloud environment. The provider manages the physical hosts; the customer manages the guest operating system and much of the workload stack.

2. What is the key difference between a private cloud and a hybrid cloud?

   Answer: A private cloud is dedicated to one organization. A hybrid cloud connects and uses private and public cloud environments together.

3. Does consumption-based billing guarantee that costs will be low?

   Answer: No. It ties charges to measured use, but waste, unexpected demand, or poor configuration can still make costs high.

4. What does “serverless” mean, and what does it not mean?

   Answer: The provider manages much of the server infrastructure so the customer can focus on application logic; it does not mean that no servers exist.

Sources

- Describe cloud computing — Microsoft Learn
- What is cloud computing — Microsoft Learn
- Shared responsibility model (AZ-900 learning unit) — Microsoft Learn
- Define cloud models — Microsoft Learn
- Describe the consumption-based model — Microsoft Learn
- Shared responsibility in the cloud — Microsoft Learn
- What are Azure Reservations? — Microsoft Learn
- Azure pricing calculator — Microsoft Learn
- Azure Functions overview — Microsoft Learn

1.2 Why organizations use the cloud

Learning intent

Describe how cloud services can help an organization maintain service availability, adjust capacity, plan performance and cost, strengthen security and governance practices, and manage resources more effectively. Understand that these are capabilities to design and use—not automatic outcomes guaranteed by choosing a cloud provider.

Core concepts

High availability and scalability (`cc-ha-scalability`). *High availability* is the design goal of keeping a service usable when failures or disruptions occur. Availability commitments are service-specific and are described in applicable service-level agreements (SLAs). An SLA is not a substitute for a resilient design: the organization must select suitable services, configure redundancy where available, and plan for recovery.

Scalability is the ability to adjust resources to meet changing demand. **Vertical scaling** changes the capacity of an individual resource—for example, selecting a virtual machine with more compute capacity. **Horizontal scaling** changes the number of resource instances, such as adding or removing application

instances. Scaling *out* adds capacity; scaling *in* removes it. *Elasticity* is the ability to adjust capacity as demand rises and falls, often by automating scale actions. Whether a particular resource can scale, how quickly it can do so, and whether the action is automatic depend on the service and its configuration.

Reliability and predictability (`cc-reliability-predictability`). *Reliability* concerns a workload's ability to recover from failure and continue to provide its intended function. A design may use redundancy or regional distribution, but cloud infrastructure alone does not ensure that an application is reliable; architecture, configuration, and recovery planning matter. *Predictability* means planning with greater confidence about workload behavior and costs. Performance predictability can benefit from monitoring, load balancing, and autoscaling. Cost predictability can benefit from measuring usage, analyzing trends, setting budgets, and estimating planned deployments. These practices inform decisions; actual demand and charges can differ from estimates.

Security and governance (`cc-security-governance`). Cloud environments provide tools and provider capabilities that organizations can use to apply security and governance controls consistently. For example, templates can standardize deployments, and auditing can help identify resources that do not match an organization's baseline. The customer must still define requirements, assign appropriate access, configure controls, protect data, and respond to findings. Governance is the system of policies and practices used to keep resources aligned with business, security, and compliance requirements; it is not a guarantee that every deployment is compliant.

Manageability (`cc-manageability`). Microsoft Learn distinguishes *management of the cloud* from *management in the cloud*. Management **of** the cloud is about operating or automating the resources themselves—for example, scaling based on demand, deploying from templates, monitoring resource health, or setting alerts. Management **in** the cloud is about the tools used to administer cloud resources, such as a web portal, command-line interface, APIs, and PowerShell. These tools can make operations repeatable and visible, but teams still need roles, processes, monitoring, and skills to use them effectively.

How it works, distinctions, limits, and defaults

Availability, reliability, scalability, and elasticity are related but distinct. A scalable service can still have a single point of failure; replication may improve availability without increasing processing capacity. Reliability includes failure recovery, high availability focuses on maintaining access, and elasticity describes capacity changing with demand.

Predictability can mean anticipating performance as demand changes or forecasting cost. Scaling can support both, but does not replace monitoring. Autoscaling rules, limits, startup time, and billing effects are service-specific; estimates use assumptions, while bills reflect actual usage and current prices.

Cloud security is shared: the provider operates underlying facilities and offers security capabilities, while the customer decides who can access data and how workloads are configured. Automation improves consistency only when templates, policies, and alerts reflect the organization's requirements.

Example or worked scenario

A ticketing company can add application instances horizontally during a popular sale and scale them back afterward. Monitoring and load balancing help the team observe and distribute demand, but it must test startup times, database capacity, and failure recovery. Finance and engineering can compare estimates with usage, standardize deployments with templates, and alert operators to health problems. These practices improve visibility and control but do not eliminate outages or cost surprises.

Exam or practical application

When asked to classify a cloud benefit, look for the core action in the scenario: keeping a workload operating amid disruptions suggests high availability; recovering and continuing after failure suggests reliability; changing capacity suggests scalability; automatically following demand up and down suggests elasticity. Better estimates of operating expenses are cost predictability; confidence that a workload meets demand is performance predictability. For manageability, ask whether the scenario is automating resource operation (management **of** the cloud) or using a tool or interface to administer resources (management **in** the cloud). Do not interpret "cloud benefit" as "automatic guarantee."

Self-check questions

1. A team adds more identical application instances to handle a traffic spike. Is this vertical or horizontal scaling?

   Answer: Horizontal scaling; the team is changing the number of instances.

2. How does reliability differ from scalability?

   Answer: Reliability is about recovering from failure and continuing to function. Scalability is the ability to adjust resources to meet demand.

3. Give one example each of management of the cloud and management in the cloud.

   Answer: Automatically replacing or scaling resources is management of the cloud. Administering resources through a portal, CLI, API, or PowerShell is management in the cloud.

4. Does using a cloud provider guarantee that an organization meets its security and compliance obligations?

   Answer: No. Provider capabilities can help, but the customer must configure and operate its controls and meet its own obligations.

Sources

- Describe the benefits of using cloud services — Microsoft Learn
- High availability and scalability in the cloud — Microsoft Learn
- Reliability and predictability in the cloud — Microsoft Learn
- Security and governance in the cloud — Microsoft Learn
- Manageability in the cloud — Microsoft Learn
- Azure Well-Architected Framework — Microsoft Learn

- Overview of Cost Management — Microsoft Learn

1.3 Cloud service types: IaaS, PaaS, SaaS

Learning intent

Explain Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS); compare the responsibilities they leave with the customer; and match each type to a practical need. The central distinction is how much of the technology stack the provider operates on the customer's behalf.

Core concepts

Infrastructure as a Service (`cc-iaas`). IaaS provides computing infrastructure—such as virtual machines, storage, and networking—as services. The provider operates the physical datacenter, physical hosts, and network infrastructure; the customer has substantial control over and responsibility for the guest operating system, configuration, applications, and data. It offers the most flexibility of the three, with more operational work; it can fit a workload requiring customer control of its OS or configuration.

Platform as a Service (`cc-paas`). PaaS provides a managed platform on which the customer builds, deploys, or runs applications. The provider manages the physical infrastructure and platform components such as the operating system and runtime. The customer focuses more on application code, data, and service configuration. It can fit a team building and deploying an application without maintaining servers or the operating system.

Software as a Service (`cc-saas`). SaaS provides a ready-to-use application operated by the provider. The provider manages most of the application stack and its infrastructure; the customer typically manages its data, users, access choices, and how the application is used. Email, messaging, and productivity applications are common examples. SaaS offers quick access to a complete application, with less infrastructure and software control than IaaS or PaaS.

The table summarizes the broad pattern; exact responsibility depends on the service and configuration.

Service type	Provider primarily manages	Customer primarily manages	Typical choice when…
IaaS	Physical facilities, hosts, and underlying network	Operating system, much of configuration, applications, data, and access	You need control over the operating system or want to move a customizable workload.
PaaS	Infrastructure, operating system, runtime, and managed platform	Application code, data, and service-level configuration	You want to build or run an application without operating its platform.
SaaS	Infrastructure, platform, and ready-made application	Data, users, access settings, and appropriate use	You need a complete application rather than a platform to build on.

Service-type use cases (`cc-service-type-use-cases`). Ask what the organization wants to manage. Choose IaaS when retaining lower-level control—especially over the operating system—is important and the team can take on patching and maintenance. Choose PaaS when the priority is developing or deploying applications while delegating infrastructure and platform upkeep. Choose SaaS when the requirement is to

use a complete application with minimal infrastructure management. Real solutions can combine service types; a single organization can use SaaS for collaboration, PaaS for a custom application, and IaaS for a legacy workload.

How it works, distinctions, limits, and defaults

Think of a stack: facilities and hardware, virtualization and networking, operating system and runtime, application, then data and user configuration. Moving from IaaS through PaaS to SaaS generally shifts more operation to the provider and reduces customer infrastructure work, but also reduces customization of provider-managed layers. Customers still have duties in every model.

IaaS differs from on-premises hosting because the provider operates the physical host and datacenter even while the customer manages a VM's OS. PaaS is distinguished by provider-managed platform components; SaaS is a provider-operated application. Across all three, customers manage data and accounts and must understand applicable access, network, and security controls. SaaS reduces infrastructure work, not identity or data-governance duties; IaaS does not place the physical datacenter under customer control.

> **Verify:** Individual cloud services may combine features or expose configuration choices that change the practical responsibility boundary. Consult the service's current documentation and responsibility guidance rather than inferring every control solely from the label IaaS, PaaS, or SaaS.

Example or worked scenario

A nonprofit wants to launch a public-facing donation website. Its developers are comfortable maintaining application code but do not want to patch a web server operating system. A PaaS web-hosting service is a plausible fit: the provider operates the underlying platform, while the nonprofit focuses on its application and data. The nonprofit still configures access, protects donor information, and monitors the application.

An older finance application needing a custom OS component may fit IaaS, while staff email and document collaboration may fit SaaS. The nonprofit can use different service types for different workloads rather than choosing a single label for the organization.

Exam or practical application

Use the responsibility boundary to classify service types: customer-managed guest OS on a cloud VM indicates IaaS; provider-managed OS/runtime with customer-deployed code indicates PaaS; a provider-operated business application indicates SaaS. Minimal infrastructure work points to SaaS for a complete product or PaaS for a customer's own app. OS control points to IaaS.

Self-check questions

1. Which service type best fits a development team that wants to deploy its own code but not patch the operating system?

   Answer: PaaS, because the provider manages the operating system and managed platform while the team focuses on its application.

2. In IaaS, who typically patches and maintains the guest operating system?

Answer: The customer. The provider manages the physical infrastructure, but the guest OS is part of the customer-managed workload.

3. A company uses a hosted email product without operating its own mail servers or application. Which service type is this?

   Answer: SaaS, a ready-to-use application operated by the provider.

4. Does choosing SaaS remove every customer security responsibility?

   Answer: No. Customers continue to manage their data, identities, users, access settings, and other controls applicable to their use of the service.

Sources

- Describe cloud service types — Microsoft Learn
- Describe Infrastructure as a Service — Microsoft Learn
- Describe Platform as a Service — Microsoft Learn
- Describe Software as a Service — Microsoft Learn
- Shared responsibility in the cloud — Microsoft Learn

CHAPTER 2

Azure architecture and services (35-40%)

This chapter maps the physical locations, organization, and services that make up Azure to everyday design choices. It introduces compute and networking, storage and migration, and identity and security concepts, emphasizing how to select a service based on a workload rather than memorize product names in isolation. Examples focus on the distinctions that help you explain a design and recognize the appropriate Azure building block.

2.1 Azure's physical and organizational structure

Learning intent

Be able to describe where Azure resources run and how Azure organizes them for management. Distinguish regions, region pairs, availability zones, and datacenters; then explain how resources, resource groups, subscriptions, and management groups form a hierarchy. The core practical idea is that physical resilience and management scope are related design concerns, but they are not the same thing: a governance hierarchy does not itself make an application highly available.

Core concepts

Regions (`as-regions`). An Azure region is a geographic area containing one or more datacenters connected through a low-latency network. A region is a deployment location for regional Azure resources; which services, features, and capacities are available varies by region. The region selected for a resource can affect latency to users and other components, data residency, resilience options, and cost. A region name in the portal is a service location choice, not a guarantee that every Azure service is available there.

Some regions are associated with a second region in a *region pair*. Some services use the pair for geo-replication or disaster recovery, and Azure may stagger updates or prioritize recovery across the pair. Deployment in one member does not itself replicate an application or automatically fail it over. Many regions are unpaired, and some services offer geo-redundancy independently. Plan application recovery explicitly and verify each service's behavior.

A *geography* is a broad market and data-residency area that can contain multiple regions. Specialized or sovereign cloud environments address particular legal, compliance, or operational requirements. A public-cloud region name alone does not establish that a specific legal regime applies; confirm requirements and service availability with current Microsoft documentation and your compliance team.

Availability zones (`as-availability-zones`). In a region that supports them, availability zones are physically separate groupings of one or more datacenters. Each zone has independent power, cooling, and networking, with connectivity designed for low-latency communication across the region. Zones are intended to reduce the chance that a localized datacenter or infrastructure failure affects every copy of a workload. Not every region or Azure service supports zones, and the service may use a different subset or count of zones than the region offers.

A *zonal* resource is placed in one selected zone; a *zone-redundant* resource is distributed or replicated across multiple zones by the service. Zonal placement alone does not protect against failure of that zone: the solution needs instances in other zones and a failover design. Service-managed zone redundancy behavior varies. A resilient web tier can still depend on a single-zone database or network component.

Datacenters (`as-datacenters`). A datacenter is a physical facility that houses computing, networking, and storage hardware. Azure datacenters are operated as part of Microsoft's cloud infrastructure; customers usually consume Azure services rather than select a specific building. Availability zones abstract physical placement into a failure-isolation choice. For a service that does not expose zone controls, consult its reliability documentation instead of assuming that a resource is in one particular datacenter or zone.

Resources and resource groups (`as-resources-rgs`). A *resource* is a manageable Azure item, such as a virtual machine, virtual network, storage account, database, or web app. A *resource group* is a logical container for resources commonly managed together. It gives a shared scope for access control, policy, monitoring, tagging, and lifecycle operations. Group resources with a sensible common lifecycle; shared networking, for example, may need a different lifecycle from an application.

Every resource belongs to a resource group, and resource groups belong to one subscription. The resource group's location stores metadata about the group; the resources within it can be in other regions. Deleting a resource group is a consequential lifecycle action because it can delete resources in the group. Use deliberate naming, access controls, and safeguards before applying bulk operations.

Subscriptions (`as-subscriptions`). An Azure subscription is a management and deployment boundary for Azure resources and is associated with a billing arrangement. It provides a scope for access control, quotas, and governance policies. Organizations often separate subscriptions to reflect environments, teams, business units, or compliance boundaries, but a subscription is not the same thing as a Microsoft Entra directory and should not be confused with a resource group. A tenant can be associated with multiple subscriptions; a subscription trusts one Microsoft Entra tenant at a time for its identity directory context.

Subscriptions also have service-specific quotas, which may be adjustable or fixed; available capacity varies by region and resource type. A subscription helps organize and govern resources but does not by itself isolate their network traffic. Check service limits before large deployments.

Management groups (`as-management-groups`). Management groups provide a hierarchy above subscriptions for organizing and governing them at scale. Policies and role assignments can be applied at a management-group scope and inherited by descendant scopes, including subscriptions and their resources, subject to the feature and scope involved. This enables central administration without configuring every subscription independently. Subscriptions in one management-group hierarchy must use the same Microsoft Entra tenant.

Resource hierarchy (`as-resource-hierarchy`). The main governance hierarchy is:

`Microsoft Entra tenant (directory) → management groups → subscriptions → resource groups → resources`

Not every organization needs every optional level. Management groups may contain child management groups or subscriptions; a subscription contains resource groups; a resource group contains resources. Parent-scope policy and role assignments can apply down the tree. A resource group does not contain a subscription, and a management group is not a physical location. This hierarchy supports governance; region and zone placement describe where supported service resources run.

Scope	Primarily helps you organize or control	Typical use
Management group	Multiple subscriptions	Apply broad policy or access governance across a portfolio
Subscription	A deployment, quota, and billing-related scope	Separate production from development or isolate cost/governance boundaries
Resource group	Related resources with a common management lifecycle	Deploy and manage one application environment
Resource	An individual service instance	Configure a VM, storage account, network, or database

How it works, distinctions, limits, and defaults

Keep physical placement (region → zones → datacenters) separate from governance (tenant → management groups → subscriptions → resource groups → resources). A resource has a place in both maps. Region pairs do not guarantee application replication or automatic failover; availability zones protect against some zone failures but are not regional disaster recovery. The component that detects failure, replicates data, routes traffic, and restores service depends on the service and architecture.

Resource Manager is Azure's consistent management layer across portal, CLI, PowerShell, APIs, and SDKs. Resource groups and subscriptions are management scopes; regions and zones are service locations. Region lists, service availability, quotas, and residency rules change, so validate them before deployment.

> **Verify:** Check the current region list, zone support, restricted-region access, and service reliability guidance.

Example or worked scenario

A retailer separates production and nonproduction into subscriptions. A management group applies a region policy to both; a production scope adds stricter rules. The production subscription has separate resource groups for its ordering app and shared networking. The app uses a supported zone-resilient tier, while the team separately configures and tests database recovery in another region. Scopes govern and organize; they do not perform failover.

Hands-on activity

In the portal, inspect **Subscriptions**, **Resource groups**, and—if permitted—**Management groups**. Compare a group's location with the regions of its resources; avoid making changes. Use the current Microsoft Learn region list to confirm service and zone availability. Optional read-only CLI: `az account show --output table` shows the active subscription; `az group list --output table` lists visible groups.

Exam or practical application

For location scenarios, identify the requirement: latency, residency, a zone failure, or regional disaster recovery. For governance scenarios, select a scope that applies to the intended subscriptions or resources. Common traps are assuming a region pair automatically deploys a replica, treating a resource-group location as the location of all its resources, or confusing a management group with a physical location.

Self-check questions

1. **Does deploying a VM in one region of a region pair automatically create a copy in the paired region?**
   Answer: No. Pairing can support some service-level replication and recovery strategies, but deployment and application failover must be configured and tested.

2. **What is the difference between a zonal and a zone-redundant resource?**
   Answer: A zonal resource is assigned to one selected zone. A zone-redundant resource is distributed or replicated across multiple zones by the service; exact behavior depends on the service.

3. **Can a resource group contain resources deployed to different Azure regions?**
   Answer: Yes. The resource group's own location concerns its metadata; it does not force every resource in the group to use that region.

4. **Which scope is above subscriptions and is used to govern several subscriptions together?**
   Answer: A management group.

5. **Put these scopes in parent-to-child order: resource group, management group, resource, subscription.**
   Answer: Management group → subscription → resource group → resource.

Sources

- Azure regions, region pairs, and nonpaired regions
- What is Microsoft Sovereign Cloud?
- List of Azure regions
- What are Azure Availability Zones?
- What is Azure Resource Manager?
- Manage resource groups with the Azure portal
- Organize resources with management groups
- Azure subscription and service limits, quotas, and constraints

2.2 Compute and networking

Learning intent

Choose a compute model that matches how much infrastructure control an application needs, and identify the foundational VM resources that make a virtual machine usable. Explain the purpose of common VM availability and scaling options, app-hosting services, virtual networks and subnets, peering, DNS, VPN Gateway, ExpressRoute, and public/private endpoints. Focus on the workload requirement—control, portability, event response, managed operations, connectivity, or exposure—rather than assuming that one service is universally best.

Core concepts

Compute types (`as-compute-types`). Azure offers multiple ways to run software. The level of control generally trades off against the amount of infrastructure the customer must manage.

Model	What it runs	Who manages most of the host environment?	Good fit
Virtual machines	A guest operating system and software selected by the customer	Customer manages the OS and installed software; Azure manages physical infrastructure	Existing software, custom OS or network needs, lift-and-shift workloads
Containers	An application packaged with its dependencies in an image	Depends on the hosting service; a managed container platform can abstract hosts and orchestration	Portable services, microservices, repeatable packaging
Functions	Small units of code activated by triggers or events	Azure manages the execution infrastructure according to the selected hosting plan	Event-driven tasks, scheduled work, or small APIs

A VM is a virtual computer with a guest operating system. It provides broad control but still requires maintenance such as patching, configuration, and software management. Containers package an app and its dependencies, but containers are not themselves a complete hosting service: choose a platform that runs and scales them. Functions let a developer focus on event-triggered code, while the platform handles much of the hosting and scaling work. “Serverless” means the customer does not manage the underlying servers in the same way; it does not mean that servers do not exist or that an application has no configuration, costs, or operational responsibilities.

VM options (`as-vm-options`). A standalone VM is suited to a workload needing a particular machine configuration. A *Virtual Machine Scale Set* manages a group of VMs and can scale the instance count based on demand or a schedule; load balancing and distribution features support applications that run across instances. An *availability set* logically distributes VMs across fault and update domains to reduce correlated maintenance or infrastructure failures, especially in regions without zones. Where supported and appropriate, Microsoft recommends considering scale sets with flexible orchestration for broad high-availability and scale capabilities; availability sets remain a distinct pattern. Neither an availability set nor a scale set guarantees application-level resilience if the application or data tier is a single point of failure.

Azure Virtual Desktop is a desktop and app virtualization service, not just a general-purpose application web-hosting option. It delivers Windows desktops or individual RemoteApps to users from host pools, including pooled or personal desktop approaches. It is useful when users need centrally managed remote desktops or published applications. It has its own service configuration, identity, networking, and VM cost considerations.

VM resources (`as-vm-resources`). A VM typically relies on a virtual network, a network interface attached to that network, an operating-system disk, and often one or more data disks. It can also use private and/or public IP addressing, network security groups (NSGs), monitoring, and identity or access configuration. A VM size determines a supported combination of compute, memory, and networking capacity; available sizes and features vary by region and workload. The OS image supplies the operating system, and credentials or keys are needed for administrative access. Consider data-disk separation where it suits the workload, and select a disk type and size for its performance and durability requirements. Each associated resource can have separate pricing or limits.

App hosting (`as-app-hosting`). Common choices include Azure App Service, container hosting, and VMs. App Service is a managed platform for web applications, mobile back ends, and REST APIs. It supports multiple language runtimes and custom containers while abstracting much of the server administration. Azure Container Apps is a managed platform for containerized applications, including APIs, microservices, jobs, and event-driven workloads, with platform features for ingress and scaling. Azure Kubernetes Service (AKS) offers managed Kubernetes when the application needs Kubernetes APIs and orchestration control; it is not identical to a platform that hides Kubernetes details. VMs offer the greatest control but require the most guest-OS operations. Azure Functions is a separate event-driven/serverless compute choice, even though functions can also run on some container platforms.

Hosting choice	Control and management emphasis	Typical scenario
App Service	Managed web-app platform; less host management	Web app or REST API using a supported runtime
Container Apps	Managed container platform with scaling and application-focused features	Containerized API, microservice, background job
AKS	Managed Kubernetes control plane; customer works with Kubernetes concepts	Kubernetes orchestration, portability or fine-grained cluster requirements
Virtual machines	Customer controls guest OS and software	Legacy or specialized software requiring machine-level control
Functions	Event-driven code with triggers and bindings	React to queue, timer, HTTP, or other supported events

Virtual networks, subnets, and peering (`as-vnet-subnets-peering`). An Azure Virtual Network (VNet) is a logically isolated network boundary for Azure resources. It has an address space defined with IP address ranges and can be segmented into *subnets*. Subnets group resources and support traffic controls or service-specific configuration; they are not necessarily separate VNets. Address ranges must be planned to avoid overlap with connected networks, especially when peering or hybrid connectivity is planned.

VNet peering connects two VNets over Microsoft's network so their resources can communicate using private IP addresses, subject to routes, security rules, and peering settings. Local peering links VNets in the same region; global peering connects across regions. Peering is not the same as merging subscription ownership or removing the need for network security. VNet peering is not transitive by default: if VNet A peers with B and B peers with C, that alone does not mean A can reach C. Centralized transit requires an appropriate hub-and-spoke or other routing design.

DNS, VPN Gateway, and ExpressRoute (`as-dns-vpn-expressroute`). DNS translates names into IP addresses. Azure DNS provides public DNS hosting and private DNS zones for name resolution in virtual networks; Azure DNS Private Resolver can support resolution between Azure private zones and on-premises DNS environments without operating DNS VMs. DNS resolves names; it does not itself create a network route or grant access.

Azure VPN Gateway connects VNets or connects an Azure VNet with an on-premises network using encrypted VPN tunnels. Common patterns include site-to-site connections for network-to-network, point-to-site connections for individual remote clients, and VNet-to-VNet connections. These VPN connections use the public internet as transport for encrypted tunnels in common hybrid setups. ExpressRoute extends an on-premises network to Microsoft cloud through a connectivity provider and a private connection that does not traverse the public internet. It can offer more consistent network characteristics for eligible use cases, but it requires connectivity-provider arrangements and appropriate circuit and routing design. Neither VPN nor ExpressRoute automatically configures the application's DNS, routes, or security rules.

Need	Typical service	Key distinction
Publish and resolve public DNS records	Azure Public DNS	DNS hosting for internet-facing domains
Resolve private names within connected VNets	Azure Private DNS	Private zone and VNet-link configuration
Encrypted connection over internet	VPN Gateway	Tunnel-based connectivity; site-to-site or point-to-site patterns
Provider-backed private connectivity	ExpressRoute	Private connection to Microsoft cloud, not a VPN tunnel over the public internet

Public and private endpoints (`as-endpoints`). A *public endpoint* is reachable through a public IP address and the public network path, subject to the service's authentication and network controls. A *private endpoint* is a network interface with a private IP address from a VNet subnet, connected through Azure Private Link to a supported service. It lets a client reach that service through a private IP path. Private endpoints require correct DNS configuration for clients to resolve the service name to the intended private address, and the service owner may need to approve the connection.

Don't confuse private endpoints with *service endpoints*. A service endpoint extends VNet identity and routes traffic to a supported Azure service over the Azure backbone, while the service continues to use its service endpoint address; a private endpoint places a private IP network interface into your VNet for the service connection. Microsoft recommends Private Link/private endpoints for many private access scenarios, but the correct option depends on architecture and service support. A private endpoint does not by itself disable every public path to the service; configure the service's network access controls as required.

How it works, distinctions, limits, and defaults

Choose compute by balancing control with managed operations. VMs allow guest-OS control but require patching and configuration; App Service reduces server management for supported web workloads; managed container platforms fit containerized apps; Functions fit event-triggered code. These are guidelines, not rigid rules. High availability and autoscaling differ: scaling changes instance count, while resilience depends on fault isolation, application behavior, data replication, and traffic distribution. Availability-set, scale-set, and zone capabilities vary by service and region.

For networking, distinguish DNS name resolution, routing, traffic filtering, and connectivity. A DNS record is not a route; a VNet or private endpoint does not grant authorization. Peering is not transitive by default. A private endpoint requires appropriate DNS and service network controls.

> **Verify:** VM sizes, zones, and supported endpoint services change; check current service guidance.

> **Conflict:** The Azure Virtual Desktop overview says the classic service retires on September 30, 2026, but that date has passed as of October 11, 2026 and the page does not confirm the post-retirement state. Verify current lifecycle and migration guidance before relying on classic resources.

Example or worked scenario

A retailer initially moves a legacy Windows app to VMs because it needs OS control. VMs sit in a VNet with separate application and database subnets and narrowly scoped NSG rules. The modernized public API later moves to App Service or a managed container platform; a file-upload process may use Functions. Branch offices connect through VPN Gateway, while ExpressRoute is evaluated for a private provider connection. Each component is matched to its own needs.

Hands-on activity

In the portal, inspect a VNet's address space, subnets, peerings, and DNS settings, then inspect a VM's size, NIC, disks, and IP configuration (read-only). Optional CLI: `az network vnet list --output table` and `az vm list-sizes --location <region> --output table` list visible VNets and available VM sizes. For a lab, use a disposable subscription and remove resources when finished.

Exam or practical application

Match workload clues: guest OS control → VM; managed web app → App Service; containerized app without Kubernetes control requirements → Container Apps; Kubernetes API → AKS; event trigger → Functions. For networking, decide whether the problem is name resolution, VNet connectivity, encrypted hybrid access, private cloud-service access, or public exposure. VM basics include a VM, network interface/VNet, OS disk, and commonly optional data disks and security controls.

Self-check questions

1. **Which compute option gives you the most control over the guest operating system?**
   Answer: An Azure virtual machine; the customer still has to configure and maintain its guest OS and software.

2. **When might you choose Azure Functions rather than a VM?**
   Answer: For code that responds to events or triggers, such as a scheduled task or file-upload processing, when a serverless execution model fits.

3. **Does peering VNet A to B and B to C automatically provide transitive connectivity from A to C?**
   Answer: No. Peering is not transitive by default; a suitable routing or transit design is needed.

4. **How does a private endpoint differ from a public endpoint?**
   Answer: A private endpoint uses a private IP address in a VNet to connect to a supported service through Private Link. A public endpoint is addressed through a public network endpoint, with access governed by service controls.

5. **Which service is commonly used for a private connectivity-provider circuit between an on-premises network and Microsoft cloud?**
   Answer: ExpressRoute. A VPN Gateway instead establishes encrypted VPN tunnels, commonly over the public internet for hybrid connectivity.

Sources

- Choose an Azure compute service
- Overview of virtual machines in Azure
- Azure Virtual Machine Scale Sets overview
- Availability sets overview
- What is Azure Virtual Desktop?
- Overview of Azure App Service
- Azure Container Apps overview
- Azure Functions overview
- What is Azure Virtual Network?
- Azure Virtual Network peering
- Azure DNS overview
- About Azure VPN Gateway
- Azure ExpressRoute overview
- What is a private endpoint?
- Azure virtual network service endpoints

2.3 Storage and data movement

Learning intent

Match a data workload to the right Azure Storage service, access tier, redundancy pattern, account type, and data-movement tool. Understand that these are separate choices: the service describes how an application works with the data; the tier expresses access economics for supported blob data; redundancy

describes the copies and failure domains; and an account type governs supported services and performance characteristics. For moving data, distinguish network copy utilities, file synchronization, migration planning, and offline bulk transfer.

Core concepts

Azure Storage services (`as-storage-services`). Azure Storage groups several managed storage services behind storage accounts, while VM disks and some specialized services have their own resource models. Select the service based on the data shape and access pattern.

Service	Data model and common purpose	Typical clue
Blob Storage	Object storage for unstructured text and binary data; supports data-lake capabilities	Images, backups, logs, documents, analytics data
Azure Files	Managed file shares accessible through SMB and, for supported configurations, NFS	Shared folder or application expects a file-system share
Queue Storage	Messages that decouple application components or hold asynchronous work	A worker needs to process a backlog independently
Table Storage	Schemaless NoSQL key/attribute data	Simple structured entities without a relational database requirement
Managed disks	Block storage volumes for Azure VMs	OS or data disk attached to a VM

Blob Storage stores objects in containers and is not a conventional mounted file share. Azure Files provides shared file-system semantics and can be mounted by clients. Queue Storage is a message store, not a general document database. Table Storage is a NoSQL option, not a relational SQL database. Managed disks are block-level storage used with VMs and are managed by Azure; they are distinct from blobs even though Azure Storage as a platform also offers Blob Storage. Some closely related services (such as Azure Elastic SAN and Azure NetApp Files) serve more specialized storage needs.

Storage access tiers (`as-storage-tiers`). Access tiers help tune Blob Storage costs according to access frequency and latency expectations. The Hot tier is for data read or changed frequently; it typically has higher storage cost and lower access cost. Cool is for infrequent access with an online, immediately available retrieval model, and Cold is for rarer access while still online. Archive is an offline tier for rarely accessed data where retrieval latency of hours can be acceptable; data generally must be rehydrated before normal online reads. Moving data between tiers, early deletion, retrieval, and transaction costs can affect total cost, so compare the full workload rather than storage price alone.

Tier	Access pattern	Online availability	Cost trade-off and planning point
Hot	Frequent access or active data	Yes	Higher storage cost, lower access costs
Cool	Infrequent access	Yes	Lower storage and higher access costs; minimum-retention rules apply
Cold	Rare access, but fast retrieval is still required	Yes	Lower storage than cool, higher access costs; minimum-retention rules apply
Archive	Very infrequent access and flexible retrieval time	No (offline)	Lowest-cost storage tier among these in many comparisons; rehydration and retention conditions matter
Smart tier	Data's hot/cool/cold usage pattern is managed automatically where supported	Tier dependent	Platform-selected movement can simplify optimization; verify availability and costs

Tier availability and supported blob types depend on account and feature configuration. Archive is not interchangeable with cool/cold: the online tiers are immediately accessible, whereas archived data needs a rehydration step. Access tiers are a Blob Storage data-management choice, not a replacement for choosing an account redundancy option.

> **Verify:** The current Microsoft Learn tier page includes Hot, Cool, Cold, Archive, and Smart tier terminology; availability, supported account types, retrieval behavior, and exam-guide emphasis can change, so check the current page and service region before using newer tier names in a design.

Storage redundancy (`as-storage-redundancy`). Azure Storage maintains copies of data for durability and availability, with selectable redundancy options. The key dimensions are whether copies stay within one datacenter or span zones in the primary region, whether data is replicated to a secondary region, and whether reads from that secondary region are available before failover.

Option	Replication pattern	Main protection	Secondary-region read access?
LRS (locally redundant storage)	Multiple copies in one physical datacenter in the primary region	Some local hardware failures	No secondary region
ZRS (zone-redundant storage)	Synchronous copies across availability zones in the primary region	A failure affecting one zone	No secondary region
GRS (geo-redundant storage)	LRS in primary region plus asynchronous replication to a secondary region	Regional disaster, after/with failover behavior	Not by default before failover
RA-GRS (read-access geo-redundant storage)	GRS replication plus read access to the secondary region	Regional disaster with secondary reads available	Yes
GZRS (geo-zone-redundant storage)	ZRS in primary region plus asynchronous replication to a secondary region	Zone failure in primary and regional disaster	Not by default before failover
RA-GZRS	GZRS plus read access to the secondary region	Zone and regional failure with secondary reads	Yes

LRS is generally the lowest-cost option and protects against local hardware failures but not the loss of the datacenter. ZRS spans zones within the primary region and supports continued service through some zone failures. GRS and GZRS add asynchronous replication to another region; an outage may mean the secondary copy is behind the primary. The `RA-` variants allow read access to the secondary region even when it has not become primary. Exact supported options vary by storage type, region, and account kind.

Redundancy is not backup. If an application or user deletes or overwrites data, those changes can be replicated to other copies. Use suitable backup, versioning, soft-delete, or recovery arrangements for accidental deletion and logical corruption. For regional disaster recovery, plan how to redirect application traffic and verify service-specific failover; replication by itself does not switch every dependent component to the secondary region.

Storage account types (`as-storage-account-types`). A storage account provides a namespace and shared configuration for storage resources such as blobs, files, queues, and tables. Standard general-purpose v2 is the recommended account type for many common scenarios because it supports these core services. Premium accounts are specialized for particular performance and storage models, including block blobs, file shares, or page blobs; they do not necessarily support every service available in general-purpose v2. The account type affects the supported redundancy choices and pricing. Choose it from workload needs and validate the feature matrix before deployment.

An account-level redundancy setting applies across storage services in that account. If different data needs different resilience or performance settings, separate accounts may be appropriate. Some storage account properties and types cannot simply be switched in place; migrating data may require creating a new account and copying data.

> **Verify:** Do not rely on remembered account labels or assume that premium, legacy, and general-purpose offerings have identical feature support; consult the current account-type comparison before creating an account.

File movement tools (`as-file-movement`). Choose a transfer method based on dataset size, network capacity, direction, and whether the activity is a one-time copy or ongoing synchronization.

Tool	Best suited to	How to think about it
AzCopy	Scriptable command-line copy or synchronization to, from, and between supported Azure Storage locations	Useful for controlled transfers and automation over a network
Azure Storage Explorer	Graphical browsing and management of supported storage resources	Useful for interactive upload, download, and inspection; it is a client tool, not a migration planning service
Azure File Sync	Keeping Windows Server file caches synchronized with a central Azure file share	Hybrid file-server solution; can tier infrequently used data to the cloud
Azure Storage Mover	Managed high-scale file migration to Azure Storage	Designed to orchestrate, monitor, and scale transfers for supported scenarios

AzCopy transfers data; Storage Explorer gives an interactive graphical interface; File Sync keeps file shares coordinated between servers and Azure Files. They solve related but different problems. File Sync is not a general-purpose migration inventory service and does not synchronize arbitrary Blob containers like a file share. Check identity, permissions, network reachability, and supported source and target types before a transfer.

Migration options (`as-migration-options`). Azure Migrate helps discover and assess workloads, build a migration plan, and coordinate migration using tools for servers, databases, web apps, and other workloads. It supports migration and modernization pathways; it is not the same product as a file-copy command. Azure Data Box addresses large offline data transfers: Microsoft ships a secure device to the customer, data is copied locally, and the device is returned for upload or export processing. It is helpful when network bandwidth is limited or a dataset is too large for practical online transfer. The specific device models, capacity, availability, and order process change over time.

How it works, distinctions, limits, and defaults

Choose storage first by data shape, then access pattern, redundancy, and transfer method. Blob is object storage; Files is a shared file system; Queue stores messages; Table stores NoSQL entities; managed disks provide VM block volumes. For supported blobs, tiers balance storage charges against retrieval, transaction, and latency costs. Archive is offline, unlike online Hot, Cool, and Cold.

Redundancy is not backup: replicated deletions or overwrites can affect copies. Zone replication addresses failures within a primary region; geo-replication adds an asynchronous secondary copy, and application failover still needs planning. Account-type, service, tier, and redundancy compatibility varies. Azure Migrate assesses and coordinates workload migration; Data Box transfers bulk data offline; AzCopy copies data; File Sync maintains hybrid file shares.

> **Verify:** Check current tiers, account-type support, regional availability, pricing, and Data Box device lifecycle.

Example or worked scenario

A media company stores video objects in Blob Storage, shared production folders in Azure Files, processing messages in Queue Storage, and VM disks as managed disks. Hot data moves to cooler online tiers when reads decline; only material that can tolerate offline retrieval is considered for Archive. File Sync keeps office caches connected to an Azure file share. For resilience, the company selects zone or geo-redundancy based on the failure it must survive and separately protects against deletion. Data Box may support an initial bulk import; File Sync or AzCopy serves different follow-on needs.

Hands-on activity

Inspect a test or authorized storage account in the portal. Record its account type, region, redundancy, and network controls; then inspect the access tier of a test blob. Do not modify production settings. Optional read-only CLI: `az storage account list --output table` and `az storage account show --name <account-name> --resource-group <resource-group> --output json`. Use a test container and approved identity for any transfer.

Exam or practical application

Select by data model: shared folder → Azure Files; unstructured object → Blob; asynchronous message → Queue; NoSQL key/attribute data → Table; VM volume → managed disk. Frequent blob access suggests Hot; immediate but infrequent access suggests Cool or Cold; offline flexible-latency data may suit Archive. LRS stays in one datacenter, ZRS spans zones, GRS/GZRS add a secondary region, and RA variants add secondary read access. Distinguish replication from backup.

Self-check questions

1. **Which service fits an application that needs a shared file share mounted by multiple clients?**
   Answer: Azure Files, using a supported protocol such as SMB or NFS for the scenario.

2. **Is Archive an online Blob access tier for immediate retrieval?**
   Answer: No. Archive is offline; data must be rehydrated before normal online access.

3. **What is the key difference between ZRS and GZRS?**
   Answer: ZRS copies data across availability zones in the primary region. GZRS combines zone redundancy in the primary region with asynchronous replication to a secondary region.

4. **Which option adds read access to a geo-replicated secondary region?**
   Answer: The read-access variants, RA-GRS or RA-GZRS.

5. **A company needs continuous synchronization between Windows file servers and a central Azure file share. Which tool is designed for this?**
   Answer: Azure File Sync.

6. **Which product helps plan and assess workload migration, and which option physically ships bulk data?**
   Answer: Azure Migrate helps assess and plan workload migration; Azure Data Box uses a shipped device for offline bulk transfer.

Sources

- Introduction to Azure Storage
- Storage account overview
- Data redundancy in Azure Storage
- Access tiers for blob data
- What is Azure Files?
- Introduction to Azure Queue Storage
- Introduction to Table storage
- Overview of Azure Disk Storage
- Copy or move data by using AzCopy v10
- What is Azure File Sync?
- About Azure Migrate
- Microsoft Azure Data Box overview
- Introduction to Azure Storage Mover
- Get started with Storage Explorer

2.4 Identity, access, and security

Learning intent

Explain how Microsoft Entra ID and Microsoft Entra Domain Services support different identity needs; distinguish authentication from authorization; recognize single sign-on (SSO), multifactor authentication (MFA), and passwordless methods; and describe external identities, Conditional Access, Azure RBAC, Zero Trust, defense in depth, and Microsoft Defender for Cloud. The key is to layer controls: verify identities, grant only the necessary permissions, evaluate context, protect workloads and data, and monitor for risk.

Core concepts

Directory services (`as-directory-services`). Microsoft Entra ID is Microsoft's cloud identity and access management service for users, groups, applications, devices, and resources. An organization uses a Microsoft Entra tenant (directory) to manage identity objects and access policies. Azure subscriptions use a directory as their identity context, while Azure RBAC controls authorization to Azure resource scopes. Microsoft Entra ID is not simply a cloud-hosted Windows domain controller; modern cloud sign-in and access management are its primary role.

Microsoft Entra Domain Services is a managed domain service for applications that still need traditional Active Directory Domain Services (AD DS) protocols and features such as domain join, Group Policy, LDAP, and Kerberos/NTLM. It reduces the need to deploy and maintain domain controllers for supported legacy workloads. It integrates with Microsoft Entra ID, which acts as the source of authority for identities, but it is not a replacement for every AD DS deployment or for Microsoft Entra ID's modern cloud identity functions.

Service	Main purpose	Typical fit
Microsoft Entra ID	Cloud identity, sign-in, authentication, and access-policy foundation	Users and apps accessing cloud and integrated services
Microsoft Entra Domain Services	Managed domain protocols and legacy AD-compatible capabilities	Cloud workloads dependent on LDAP, Kerberos/NTLM, domain join, or Group Policy

Authentication methods (`as-authentication-methods`). Authentication answers “Who are you?”; authorization answers “What are you allowed to do?” Microsoft Entra ID supports passwords, authenticator-based sign-in, passkeys, certificate-based methods, and other methods, subject to configuration and service capability. The exact authentication method available for first factor, MFA, self-service password reset, or recovery varies; consult the current method matrix rather than assuming every method can be used for every purpose.

Single sign-on (SSO) allows a user to authenticate once and then access multiple applications without re-entering credentials for each app, subject to the applications' sign-in sessions and policies. SSO improves user experience and can reduce password prompts, but it does not remove the need to secure the sign-in session or control authorization to each resource.

MFA requires more than one verification factor in a sign-in. Factors are commonly described as something you know, have, or are. A password plus a second proof can make stolen passwords less sufficient on their own; however, not all MFA methods offer the same phishing resistance. Passwordless authentication lets a user sign in without a conventional password, using supported methods such as passkeys or authenticator mechanisms. A passwordless method can also satisfy strong authentication requirements depending on configuration. Avoid equating “passwordless” with “no authentication” or treating every method as equally resistant to phishing.

Method or concept	What it means	Why it matters
SSO	One authenticated session can be used across integrated apps	Reduces repeated sign-in prompts; app access still follows policy
MFA	More than one proof is required	Reduces reliance on a password alone
Passwordless	Sign-in without entering a conventional password	Can improve usability and reduce password-related risk; method security varies

External identities (`as-external-identities`). Microsoft Entra External ID supports secure access for people outside an organization. For business-to-business (B2B) collaboration, a partner or guest can use an external identity to collaborate in the organization's workforce tenant, with access limited to what is granted. For customer identity and access management (CIAM), an organization can use an external-

tenant configuration to manage consumer or business-customer sign-up and sign-in experiences separately from its workforce directory. The distinction is audience and tenant purpose: workforce collaboration with invited partners is not the same scenario as customer sign-in to a public-facing application.

External users should receive only the permissions needed for the collaboration or application. Invitation, federation, self-service registration, authentication method, terms, and lifecycle settings depend on the configuration. A guest identity is not automatically an administrator or a trusted user for all resources.

Conditional Access (`as-conditional-access`). Microsoft Entra Conditional Access is a policy engine that evaluates signals and applies access controls after first-factor authentication. Policies can consider the user or group, application, device, location, and risk signals, then require an action such as MFA, require a compliant device, or block access. It is often expressed as an "if this context, then that control" rule. For example, require MFA when an employee accesses a sensitive application from an unfamiliar context.

Conditional Access is not a replacement for authentication or a firewall, and it is not intended as the first-line defense against every type of attack. Policies need to be tested carefully to avoid unintended lockouts, and exclusions should be narrow and monitored. Licensing and feature availability affect which signals and controls are available.

Azure role-based access control (`as-rbac`). Azure RBAC is Azure Resource Manager's authorization system for managing access to Azure resources. A role assignment combines three parts: a *security principal* (user, group, service principal, or managed identity), a *role definition* (the allowed actions), and a *scope* (where those permissions apply). Azure built-in roles include broad roles such as Owner and narrower roles such as Reader or service-specific contributor roles; custom roles can be created when built-ins do not meet a need.

Common scopes are management group, subscription, resource group, and individual resource. A role assignment at a parent scope can be inherited by descendant resources. Use the narrowest practical scope and least-privilege role. Azure RBAC is not the same as Microsoft Entra directory roles: RBAC governs Azure resource access; directory roles administer identity and tenant functions. A user can authenticate successfully yet lack permission to create or modify an Azure resource.

Zero Trust (`as-zero-trust`). Zero Trust is a security model built around "never trust, always verify." It assumes that location on an internal network does not automatically make a request safe. The three common principles are:

- **Verify explicitly:** authenticate and authorize access using available identity, device, location, and risk context.
- **Use least privilege:** grant only the minimum access needed and only for as long as needed.
- **Assume breach:** design controls to detect compromise, limit movement and impact, and support response.

Zero Trust is not one product or a single network appliance. It is a way to combine identity, devices, network, applications, data, infrastructure, and monitoring controls. It encourages continuous evaluation as conditions change.

Defense in depth (`as-defense-in-depth`). Defense in depth uses multiple layers of security controls so that failure of one control does not immediately expose the entire system. A simplified Azure-oriented view includes physical security operated by the cloud provider, identity and access controls, perimeter and network protections, compute and application hardening, data protection, and detection/response. The cloud provider and customer share responsibilities; the exact division depends on the service model. For example, Azure operates datacenter facilities, while the customer configures resource access, workload settings, and data permissions.

The layers should be complementary, not redundant checkboxes. MFA protects sign-in; RBAC limits actions; NSGs and private endpoints constrain network exposure; encryption protects data in transit or at rest; backup supports recovery; monitoring and Defender for Cloud can help identify risk. No single layer guarantees that a workload cannot be compromised.

Microsoft Defender for Cloud (`as-defender-for-cloud`). Defender for Cloud is a cloud security platform for assessing and improving security posture and protecting workloads across Azure and supported hybrid or multicloud environments. Its capabilities include cloud security posture management (CSPM), workload protection (CWPP), and security support across development workflows. It surfaces recommendations, security findings, and threat-protection capabilities depending on environment, plan, and configuration. It helps teams identify risks and improve protections; it does not automatically remediate every finding or replace the team's secure design and operations.

How it works, distinctions, limits, and defaults

Treat identity, policy, authorization, network controls, and monitoring as distinct layers. Authentication proves identity; Conditional Access evaluates context and can require MFA or block; Azure RBAC authorizes actions on Azure resources. A user may sign in successfully yet lack permission to change a resource. Use the least-privileged principal, role, and scope. Parent-scope assignments can be inherited by descendants.

Conditional Access policies need careful testing to avoid lockouts; licensing and configuration affect signals and controls. Entra ID supports modern cloud identity, while Domain Services serves compatible legacy AD protocol needs. Defender for Cloud recommendations depend on onboarding and enabled plans and do not automatically fix every issue.

> **Verify:** Microsoft Entra naming, Defender portal experiences, plans, licensing, available authentication methods, and preview states change; consult current Learn guidance.

Example or worked scenario

A company gives supplier guests access to one project through Entra External ID B2B collaboration. It assigns a group a narrow Azure RBAC role at the project resource group and uses Conditional Access to require MFA for the application. An app uses a managed identity rather than a stored password to access a storage service. A private endpoint limits the network path, and Defender for Cloud helps surface posture recommendations. Backup and recovery remain separately planned.

Hands-on activity

In an authorized test tenant, inspect Entra **Users**, **Groups**, **Authentication methods**, and **External identities**. In the Azure portal, review a resource group's **Access control (IAM)** assignments and identify principal, role, and scope without changing them. Inspect Conditional Access policies in report-only mode where available, then review Defender for Cloud recommendations. Do not invite users, assign roles, or enable production policies as practice.

Exam or practical application

Separate authentication from authorization: SSO shares sign-in across integrated apps; MFA uses multiple proofs; passwordless omits a conventional password; RBAC governs Azure resource actions. Use Domain Services for supported legacy AD protocols, External ID for partners or customers, and Conditional Access for context-based controls. Zero Trust means verify explicitly, use least privilege, and assume breach. Defender for Cloud helps improve posture and protect supported workloads; it is not a directory, firewall, or automatic fix for all risks.

Self-check questions

1. **What are the three components of an Azure RBAC role assignment?**
   Answer: Security principal, role definition, and scope.
2. **An application needs LDAP and Kerberos but cannot be modernized. Which service may provide managed domain capabilities?**
   Answer: Microsoft Entra Domain Services, subject to workload compatibility and service requirements.
3. **Does MFA decide whether a signed-in user may delete a resource group?**
   Answer: No. MFA is an authentication control. Azure RBAC determines whether the principal is authorized for that Azure operation at the applicable scope.
4. **What are the three Zero Trust principles?**
   Answer: Verify explicitly, use least privilege, and assume breach.
5. **What is Conditional Access for?**
   Answer: Evaluating sign-in context and applying controls such as requiring MFA or blocking access according to policy.
6. **What is Microsoft Defender for Cloud's general purpose?**
   Answer: Help assess and improve cloud security posture and protect supported workloads across Azure and supported hybrid or multicloud environments.

Sources

- What is Microsoft Entra?
- Overview of Microsoft Entra Domain Services
- Microsoft Entra authentication overview
- What is single sign-on in Microsoft Entra ID?
- Microsoft Entra External ID overview

- Microsoft Entra Conditional Access overview
- What is Azure role-based access control (Azure RBAC)?
- Zero Trust as a security foundation
- Introduction to Azure security
- Microsoft Defender for Cloud overview
- Azure Virtual Network overview
- What is a private endpoint?

CHAPTER 3

Azure management and governance (30-35%)

Azure gives organizations tools to understand what cloud resources cost, apply rules consistently, deploy resources repeatably, and detect problems before they become outages. It focuses on estimating, setting guardrails, deploying consistently, and responding to health signals.

3.1 Costs in Azure

Learning intent

Learn to recognize what can change an Azure bill, create a planning estimate with the Azure pricing calculator, use Microsoft Cost Management to inspect and manage spending, and use tags to organize resources and cost data. Remember that an estimate is a planning aid, while billing data records measured and rated use; neither a tag nor a budget by itself prevents all spending.

Core concepts

Factors that affect Azure costs (`mg-cost-factors`). A cloud bill is the result of what you provision, how much you use it, and the applicable price and commercial terms. The service and configuration matter: for example, compute size, storage capacity or access pattern, database tier, and network design determine which meters apply. The quantity and duration of use matter too. A virtual machine left running for more hours generally incurs more compute usage than one stopped and deallocated sooner; storage can continue to incur charges even when a workload is idle. Transferred data, backups, monitoring, and other related services may contribute additional charges.

Location can affect the available products and their rates, so a configuration estimated in one region should not be assumed to cost the same in another. Discounts, negotiated prices, reservations, savings plans, and other eligible purchase choices can also change the effective price. Taxes, support, licensing, marketplace purchases, credits, and billing-specific arrangements may be accounted for differently from the resource-level cost view. Always identify which costs a particular estimate or report includes before using it as an invoice prediction.

A useful mental model is **configuration × measured consumption × applicable rate**, with billing adjustments and commercial terms layered on top. It is not a universal formula: products meter usage differently, and cost data may not match a finalized invoice.

Azure pricing calculator (`mg-pricing-calculator`). The Azure pricing calculator is a public, browser-based planning tool. Build an estimate by adding services, selecting product variants and region, and entering expected usage. The calculator displays the resulting estimated costs for the selected configuration. For some offerings it can show upfront and monthly amounts, and some services expose

purchase options such as reserved capacity or savings plans. Signing in can let the calculator use negotiated or discounted prices associated with the account; the displayed public estimate is not necessarily the price a particular customer will pay.

Make calculator assumptions explicit: estimate the intended instance count and size, runtime, storage, transfer, and supporting services. Revisit the estimate as the design changes. It is not a quote, deployment, or spending cap; actual costs depend on actual usage and applicable account terms.

Microsoft Cost Management (`mg-cost-management`). Microsoft Cost Management provides FinOps-oriented tools to analyze, monitor, and optimize Microsoft cloud spending. Azure users can inspect cost data at available scopes such as billing accounts, subscriptions, resource groups, and management groups, subject to their access and agreement. Cost Analysis helps answer questions such as "Which services drove this month's increase?" or "How is spending distributed across teams?" Users can group and filter data, review trends, and export cost details for other analysis or processes.

Cost Management supports proactive tracking as well as retrospective analysis. Budgets let an authorized user set a spending or usage target and configure alerts when actual or forecast cost reaches defined thresholds. An alert notifies people or systems; it does not automatically stop resources or guarantee that the final bill stays below the target. Cost anomaly features can help surface unusual patterns, and scheduled alerts and exports can support recurring review and downstream reporting. Available features, scopes, and billing data depend on the account and current service capabilities.

Keep the distinction between **Cost Management** and **Billing** clear. Cost Management is for analyzing, managing, and optimizing cost. Billing experiences address the commercial billing relationship—such as invoices, payment methods, and credits—where applicable. The data and timing are related but not identical. In particular, invoices may include credits, taxes, or purchases that are not represented in the same way in Cost Management data. If reconciliation matters, use the appropriate billing documents and scope rather than treating a cost chart as an invoice.

Tags (`mg-tags`). A tag is a key-value pair of metadata attached to a resource, resource group, or subscription. For example, `Environment = Production` or `CostCenter = 1200` can support organization, automation, reporting, and cost attribution. Where billing data supports tags, teams can group costs by department, application, or project.

Tags are not a separate access-control system, and adding a tag does not itself change a resource's behavior. A resource also does not automatically inherit tags from its resource group or subscription. An organization can use Azure Policy to require or apply tags, and Cost Management supports tag inheritance and cost-allocation capabilities in supported contexts, but the existence of a parent tag alone does not guarantee child resources or every cost record carry it. Tags are plain text: do not put secrets, personal data, or other sensitive values in them. Agree on consistent key names and value conventions so reports remain useful.

How it works, distinctions, limits, and defaults

Cost estimation and cost observation answer related but different questions. The calculator asks, "Given this proposed configuration and these assumptions, what might it cost?" Cost Management asks, "What cost data is available for the scope and period I can access, and how can I analyze or manage it?" Billing

asks, “What are the charges, credits, and payment obligations in the billing relationship?” Use each for its purpose: estimate before deployment, inspect actual cost during operation, and reconcile with billing artifacts when needed.

The main cost drivers are not limited to a monthly service price. Consider the chosen service tier and resource size, the quantity provisioned, the time it runs, usage-based measures such as requests or data processed, data storage and transfer, region, and applicable discounts or commitments. A design that reduces one meter can increase another—for example, additional redundancy may raise storage or transfer requirements while improving resilience. Cost optimization is therefore a design trade-off, not simply choosing the smallest virtual machine.

Tool or mechanism	Main question it answers	What it does not guarantee
Azure pricing calculator	What is a planned configuration likely to cost under stated assumptions?	It does not deploy resources, lock in a price, or guarantee an invoice amount.
Cost Analysis	What cost data is available for this scope and period, and how is it distributed?	It is not necessarily the final invoice or a live meter with zero delay.
Budget and alert	Has spending or a forecast reached a threshold that people should review?	An alert is not a spending cap and does not automatically stop resources by default.
Tags	How can resources and supported cost records be classified or grouped?	Tags do not enforce permissions, inherit automatically, or guarantee complete billing coverage.

Cost data and billing records can differ in timing and scope. Before reconciling differences, check the period, price terms, credits, taxes, and data definitions; do not promise an exact amount without confirming current prices and account terms.

> **Verify:** Calculator catalog options, negotiated-price behavior, Cost Management features, report scopes, and billing-data timing can change. Check the live calculator and the account's applicable Cost Management and billing experience before making a purchasing commitment.

Example or worked scenario

A small product team plans a test environment with two virtual machines, managed storage, and a database. Before creating anything, the team uses the calculator to add each service, chooses the intended region and service tiers, enters the expected hours and capacity, and checks whether any applicable purchase options should be modeled. It records two scenarios: one for a continuously running test system and another for a system shut down outside working hours. The comparison makes the runtime assumption visible instead of hiding it inside one monthly total.

After deployment, the team applies `Application = CatalogDemo`, `Environment = Test`, and `CostCenter = 1200` tags according to its naming convention. In Cost Analysis, it filters to the test resource group, compares the current period with the prior period, and groups by service or tag where available. It sets a budget with an alert for the team mailbox, but treats the message as a prompt to investigate—not as an

automatic shutoff. If actual cost is higher than expected, the team checks whether VMs ran longer, data transfer or storage grew, or the selected configuration differed from the estimate. It also checks the billing view when reconciling final charges.

Hands-on activity

Use an authorized sandbox; no deployment is required.

1. Open the Azure pricing calculator, add compute and storage, and vary region, runtime, or capacity. Record your assumptions and any costs not modeled.
2. In the portal, open Cost Analysis for an authorized scope and note the time range, grouping, and filters available.
3. Inspect budgets and alerts, if permitted, without creating one for production. Identify a sensible threshold and recipients for a test subscription.
4. Review resource tags and verify child resources actually have them; do not assume inheritance.

Exam or practical application

When a question asks which option helps **estimate future Azure use**, choose the pricing calculator. When it asks how to **analyze spending, track trends, or set threshold notifications**, think Microsoft Cost Management. When the goal is to **classify resources or group supported cost information**, think tags. If the scenario says spending must stop at a precise cap, do not assume a budget alert is an enforcement mechanism; distinguish an alert from an action that actually changes or stops a workload.

For a practical review, state the estimate's region, product configuration, quantity, runtime, and purchase assumptions. Identify the billing scope and period when reading actual costs.

Self-check questions

1. **A project wants a predeployment estimate for a planned set of Azure services. Which tool should it start with?**
   Answer: The Azure pricing calculator. It models expected products and usage; it does not deploy them or guarantee the final bill.

2. **Name three factors that can change an Azure cost.**
   Answer: Any three of service type/tier or size, quantity, runtime, usage volume, region, data transfer, storage, or applicable discounts and purchase terms.

3. **Will a Cost Management budget alert automatically stop the resources when a threshold is reached?**
   Answer: No. A budget can trigger notifications or configured alert actions, but a budget is not by itself a resource shutoff or guaranteed spending cap.

4. **What is a tag, and what is one important limitation?**
   Answer: A tag is key-value metadata used to classify resources and, in supported reports, organize costs. It does not automatically inherit from a parent resource group or subscription, and sensitive data should not be stored in it.

5. **Why might a Cost Analysis amount differ from a final invoice?**
   Answer: Cost data and invoice processing serve different purposes and can differ in timing, scope, credits, taxes, purchases, and other billing adjustments. Check the relevant billing artifacts for invoice reconciliation.

Sources

- Describe Cost Management in Azure — AZ-900 Learn module
- Overview of Cost Management
- Estimate costs with the Azure pricing calculator
- Use tags to organize Azure resources and management hierarchy
- Cost Management budgets
- Cost analysis in Cost Management

3.2 Governance and compliance

Learning intent

Distinguish the role of Microsoft Purview in data governance, protection, and compliance from the role of Azure Policy in applying resource-governance rules, and from resource locks that protect against accidental control-plane changes. Learn to match each tool to the problem it solves, and understand why none of these tools replaces identity permissions or an organization's broader compliance program.

Core concepts

Microsoft Purview (`mg-purview`). Microsoft Purview is a suite of integrated solutions that helps an organization govern, protect, and manage data across its data estate. Its broad capabilities include data governance, data security, and data compliance. Depending on the solution and configuration, Purview can help organizations gain visibility into data, manage sensitive information, address regulatory requirements, and apply data-protection practices. Examples of the product areas include Data Map and Unified Catalog for data governance, and compliance and information-protection solutions for data compliance and security.

Think of Purview in terms of **data**: understanding data assets, managing sensitive information and its lifecycle, and supporting governance and compliance processes. It does not mean every Azure resource has been configured securely, nor does it automatically make an organization compliant with a law or standard. Purview capabilities and licensing vary by solution. It is distinct from Azure Policy, which evaluates and governs Azure resources and requests against assigned policy rules.

Purview has evolved from older Azure Purview descriptions. Verify the current solution, supported data sources, and licensing for a specific catalog, security, or compliance need.

Azure Policy (`mg-policy`). Azure Policy helps enforce organizational standards and assess compliance at scale. A policy definition describes a rule, such as allowing deployments only in approved regions or requiring a tag. An administrator assigns the definition—or a group of definitions called an initiative—to a scope, such as a management group, subscription, resource group, or resource. A higher-level assignment can cover resources below it; exclusions and exemptions can be used when authorized and appropriate.

Policy rules use effects to determine the response to evaluation. Depending on the definition, a policy can audit a resource, deny a noncompliant create or update, modify a request, deploy related configuration, or support remediation of existing resources. The compliance view reports evaluation results and helps identify noncompliant resources. Some policies only report; they do not automatically repair every existing resource. Remediation behavior and permissions depend on the policy effect, assignment, and resource provider.

Policy is a way to govern **what resources are deployed or how they are configured**, and in some cases to evaluate or restrict actions in a request. It is not the same as granting a user permission to perform operations. Use Azure RBAC to manage access by assigning roles to identities at scopes. A user can have permission to make a request that Azure Policy then denies because the request conflicts with an assigned rule.

Resource locks (`mg-locks`). A management lock helps protect a subscription, resource group, or resource from accidental deletion or modification. The two common lock types are **CanNotDelete** (shown in the portal as **Delete**) and **ReadOnly** (shown in the portal as **Read-only**). A CanNotDelete lock allows authorized users to read and modify a resource but prevents deletion. A ReadOnly lock permits reads but blocks updates and deletes through the control plane.

A lock adds a restriction regardless of RBAC permissions; it grants no access. Parent locks are inherited, including by later resources, and the most restrictive applicable lock governs. Remove a lock before a blocked operation. Since ReadOnly locks can disrupt routine management calls, use them carefully.

How it works, distinctions, limits, and defaults

The following comparison is useful when multiple governance controls sound similar. They can coexist; the best answer depends on whether the problem is access, standards, protection against accidental changes, or data oversight.

Capability	Primary purpose	Typical question	Scope or target	Important boundary
Azure RBAC	Grant permissions to an identity through role assignments	Who may perform this operation?	Azure management scopes and supported data-plane scopes	Permission can be constrained by scope and role, but it does not itself define the organization's desired configuration.
Azure Policy	Evaluate and enforce resource-governance rules	Is this resource or request compliant with our standards?	Policy-supported management scopes and resources	An audit policy reports rather than blocks; effects and remediation vary by definition.
Resource locks	Prevent accidental control-plane deletion or modification	How can we guard this resource against a mistaken management operation?	Subscription, resource group, or resource	A lock does not grant access or protect every operation on the data inside a service.
Microsoft Purview	Govern, protect, and manage data; support compliance processes	What data do we have, how is it classified or protected, and how do we manage data risk?	Data estate and Purview-supported solutions	Capabilities, data sources, and licensing vary; it is not a blanket Azure-resource configuration policy.

Locks are not backups or data-plane access controls. They apply to control-plane operations, so a lock on a storage account does not necessarily prevent a data-plane user from deleting a blob. ReadOnly locks can also block management operations that use write-style API calls. Check service-specific effects before applying locks broadly.

Policy evaluates resources on relevant lifecycle and assignment events and periodically, so compliance is not always instantaneous. Existing resources might need a separate remediation task and suitable permissions; inspect the policy effect rather than assuming assignment fixes them.

> **Verify:** Microsoft Purview solutions, supported data sources, licensing, and portal names change over time. Similarly, Azure Policy effects and resource lock behavior can differ by resource provider and operation. Check the current service documentation before treating a rule or lock as a complete compliance or data-protection control.

Example or worked scenario

A company requires production resources in approved regions, cost-center tags, deletion protection for a critical network resource, and governance for sensitive data.

An administrator assigns policies for allowed locations and required tags, then reviews compliance and remediates existing resources where supported. RBAC still determines who can create or modify resources. A CanNotDelete lock adds deletion protection to the critical network resource and is removed through a controlled process when needed. The data team evaluates Microsoft Purview for supported data assets; it does not replace resource policies or locks.

Hands-on activity

Use a lab subscription or read-only browsing. Do not assign a deny policy or apply a lock to a shared or production resource.

1. In the portal, inspect built-in Azure Policy definitions for allowed locations or required tags. Read the effect and distinguish a definition, assignment, and initiative.
2. On a disposable lab resource, inspect the **CanNotDelete** and **ReadOnly** locks and their scope. Do not apply a lock to shared infrastructure.
3. Review the current Microsoft Purview overview and identify its data-governance, data-security, and data-compliance solution areas; confirm tenant availability before planning deployment.

Exam or practical application

Choose **Azure Policy** when the requirement is to assess or enforce standards such as allowed regions, required tags, or configuration rules across a scope. Choose **RBAC** when the question is about which identity can perform an operation. Choose a **resource lock** when the requirement is protection from accidental control-plane deletion or modification, particularly when even an authorized role holder should encounter an extra barrier. Choose **Microsoft Purview** when the focus is governing, discovering, protecting, or managing data and related compliance processes.

Avoid confusing “compliant” with “secure in every respect.” A policy compliance result is evidence about specified rules; a lock protects a defined set of management operations; Purview supports data governance and protection; RBAC controls access. Real governance uses these controls in combination with monitoring, data protection, process, and human review.

Self-check questions

1. **Which service is designed to apply a required-tag or allowed-region rule across a resource scope?**
   Answer: Azure Policy. An assignment evaluates resources at its scope and applies the effect defined by the policy.

2. **A developer has Contributor access, but a resource has a CanNotDelete lock. Can the developer delete it through a normal Azure management operation?**
   Answer: No. The lock adds a deletion restriction that applies even to users whose RBAC role would otherwise permit deletion. An authorized administrator must remove the lock first.

3. **What is the difference between Azure Policy and RBAC?**
   Answer: RBAC grants or limits what an identity can do. Policy evaluates or governs whether a resource or request conforms to assigned standards. A permitted RBAC action can still be denied by policy.

4. **Does a lock on a storage account guarantee that no one can delete data in that account?**
   Answer: No. Locks apply to control-plane management operations and do not generally block data-plane operations against stored data. Use appropriate data-plane access controls and protection mechanisms too.

5. **What is Microsoft Purview primarily for in this comparison?**
   Answer: It provides integrated solutions for governing, protecting, and managing data and supporting data compliance. It is not a substitute for Azure Policy's resource rules or RBAC permissions.

Sources

- Learn about Microsoft Purview
- Overview of Azure Policy
- Lock your Azure resources to protect your infrastructure
- Azure RBAC overview

3.3 Managing and deploying resources

Learning intent

Understand that Azure resources can be managed through a shared Azure Resource Manager layer using the portal, command-line tools, APIs, and templates. Distinguish the Azure portal from Azure CLI and Azure PowerShell, and distinguish those tools from Cloud Shell, which supplies a browser-accessible command environment. Learn what Azure Arc adds for hybrid and multicloud management, what infrastructure as code means, and how ARM templates and Bicep support repeatable deployments.

Core concepts

Azure portal (`mg-portal`). The Azure portal is a browser-based graphical interface for creating and managing Azure resources. It provides service blades, dashboards, search, resource views, and access to management features. It is useful for discovery, visual inspection, and one-off changes, especially while learning a service. Portal actions are requests to Azure management services; using a graphical interface does not make an action inherently safer or exempt it from access controls, policies, locks, or costs.

The portal is one management option, not a separate deployment engine. The same Resource Manager capabilities are also exposed through Azure CLI, Azure PowerShell, REST APIs, and SDKs. An administrator can choose the interface that fits a task, then use role assignments and governance rules to control what happens.

Cloud Shell, Azure CLI, and Azure PowerShell (`mg-shell-cli-powershell`). Azure CLI and Azure PowerShell are command-line management tools. The Azure CLI uses commands beginning with `az`, such as `az group list`; it is cross-platform and can be used interactively or in scripts. Azure PowerShell is the collection of Microsoft PowerShell modules for Azure; the current recommended module is **Az**, with cmdlets such as `Get-AzResourceGroup`. PowerShell cmdlets commonly return structured .NET objects that can be piped into other PowerShell commands. The CLI commonly formats output as JSON by default and supports other output formats.

Azure Cloud Shell is an authenticated, browser-accessible terminal available in the Azure portal and at shell.azure.com. It offers a Bash or PowerShell experience and comes with common Azure command-line tools. It is an **environment**, not a third command language: choose Bash or PowerShell and run Azure CLI or Az PowerShell commands in that shell. Cloud Shell is hosted and maintained by Microsoft, so it can

be convenient when you do not want to install tools locally. Cloud Shell uses persistent file storage for the user's files and also runs on a temporary host; the service documentation describes session timeout and storage requirements. The hosting machine is free to use, but the required storage account can incur ordinary storage charges.

Choice	What it is	Typical fit	Key distinction
Azure portal	Browser-based graphical management interface	Explore resources, review configuration, perform guided tasks	A GUI that sends management requests; not the only way to manage Azure.
Azure CLI	Cross-platform command-line tool with `az` commands	Shell scripts, interactive administration, cross-platform automation	Commands commonly present results as JSON and can be composed in a shell.
Azure PowerShell	Azure's PowerShell modules and cmdlets, currently the Az module	PowerShell automation and object-based pipelines	Uses PowerShell cmdlets and objects; legacy AzureRM examples are not the current module.
Azure Cloud Shell	Hosted browser shell environment with Bash or PowerShell	Run CLI or PowerShell without local installation	An environment for tools, not a replacement name for CLI or PowerShell.

Azure Arc (`mg-arc`). Azure Arc extends Azure management and governance capabilities to supported resources outside Azure, including eligible servers, Kubernetes clusters, databases, and virtualized environments. Arc represents supported non-Azure resources in Azure Resource Manager so that teams can use a familiar Azure management experience and selected Azure services across on-premises, edge, and other cloud environments. The exact management operations depend on the Arc-enabled resource type and its configuration.

Arc does not move an on-premises server into Azure or make every Azure service work identically everywhere. Onboarding, agents or extensions, network connectivity, and service prerequisites depend on resource type. Arc's control-plane representation does not change compute location, and attached Azure services can incur their own charges.

Infrastructure as code (`mg-iac`). Infrastructure as code (IaC) means defining infrastructure in machine-readable files and managing those definitions as code, commonly using source control, review, and automated deployment. Rather than repeat a sequence of manual actions, a team describes the intended resources and configuration, then uses tooling to deploy them. IaC helps with consistency, reuse, review, change history, and recreating environments. It does not remove the need for permissions, testing, secrets management, or careful review of planned changes.

IaC may be declarative or imperative; ARM templates and Bicep are declarative Azure options. Versioned files can be parameterized and reused across environments. Review changes before deployment, especially possible deletions, replacements, or exposure.

Azure Resource Manager and ARM templates (`mg-arm`). Azure Resource Manager (ARM) is Azure's deployment and management service. Requests from the portal, command-line tools, APIs, and SDKs pass through the management layer, which authenticates and authorizes requests before forwarding

them to resource providers. ARM provides consistent management capabilities such as deployment, access control, tags, and locks. Azure's management scopes include management groups, subscriptions, resource groups, and resources; scope affects how broadly settings and deployments apply.

An **ARM template** is a JSON file that declares the Azure resources and properties to deploy. Templates can describe one resource or a larger solution, including dependencies, parameters, and outputs. Resource Manager orchestrates deployment ordering for dependent resources. A template can be deployed repeatedly to help keep environments consistent. ARM deployments can also be used at different scopes when supported by the template and deployment operation.

Bicep is a domain-specific declarative language for Azure resources. It offers a more concise authoring syntax than raw ARM template JSON while targeting the same Azure Resource Manager deployment platform. Bicep files are converted to ARM templates during deployment, so Bicep is not a different control plane. When comparing ARM and Bicep, compare authoring format and workflow, not whether they can deploy Azure resources.

How it works, distinctions, limits, and defaults

Management interfaces and infrastructure definitions are complementary. The portal is convenient for exploration and guided tasks. CLI and PowerShell make commands repeatable and scriptable. Cloud Shell provides a browser-hosted place to run command-line tools. IaC files record intended infrastructure and can be reviewed or used in automation. All ultimately operate subject to identity, scope, policy, locks, provider support, and service-specific configuration.

Dimension	ARM template	Bicep
Authoring format	JSON	Azure-focused declarative language with a more concise syntax
Deployment destination	Azure Resource Manager	Azure Resource Manager; Bicep is compiled to an ARM template during deployment
Desired use	Declare resources and configuration for repeatable deployment	Declare resources and configuration with simpler authoring and type-aware tooling
Relationship	Native template format used by the deployment platform	Alternative authoring experience for ARM deployments, not a separate Azure management service

A declarative template specifies target configuration, and ARM orchestrates represented dependencies. Repeatability does not guarantee success: provider behavior, API versions, permissions, current state, and external dependencies matter. Review changes and use ARM **what-if** when appropriate.

Cloud Shell's persistence depends on its configured storage, even though its host is temporary. Cloud Shell may be subject to inactivity timeout, and local installation may be more suitable for durable development, offline work, specialized extensions, or local automation. The shell choice also matters: Bash syntax and PowerShell syntax differ, even when the Azure operation is conceptually the same. Avoid copying commands between environments without checking quoting, variables, and pipeline behavior.

> **Verify:** Cloud Shell session behavior, storage prerequisites, installed tool versions, Azure CLI and Az module versions, ARM API versions, and Azure Arc resource capabilities can change. Use live documentation and check the selected subscription and tenant before running commands that create, modify, or delete resources.

Example or worked scenario

A company wants a consistent development environment in two subscriptions and visibility into an on-premises Linux server. An engineer explores the portal, writes Bicep, reviews it in source control, previews changes, and deploys after checking subscription and scope.

An engineer uses Cloud Shell Bash and Azure CLI for investigation; another uses Az cmdlets in automation. The on-premises server is connected through Azure Arc but remains on-premises. Any attached monitoring or security service is assessed separately for prerequisites and cost.

Hands-on activity

This activity uses read-only commands until the optional template preview. Confirm the active tenant and subscription; do not run deployment commands in a production scope.

1. In the portal, inspect an authorized resource group's subscription and location.
2. In Cloud Shell Bash, run `az account show --output table` and `az group list --output table` to check context and list visible groups. In PowerShell, compare `Get-AzContext` and `Get-AzResourceGroup`.
3. Read a small ARM template or Bicep sample and identify a resource, name, location, and configuration. Use `what-if` only in a lab after verifying the deployment context.
4. From Azure Arc documentation, identify a supported non-Azure resource and its onboarding prerequisites.

Exam or practical application

If the question asks for a **browser GUI**, think Azure portal. If it asks for **commands or scripts**, Azure CLI or Azure PowerShell are both management tools; determine which syntax and shell the scenario specifies. If it asks for **a browser terminal with Bash or PowerShell and preinstalled tools**, think Cloud Shell. If it describes **governing or managing supported servers or clusters outside Azure through Azure**, think Azure Arc.

If the requirement is to create consistent environments from source-controlled definitions, think IaC. ARM templates are JSON declarations, while Bicep offers a more concise language that deploys through ARM. ARM itself is the management and deployment service, not merely the JSON template file. Make sure you distinguish the platform, a template format, the command-line tool, and the environment in which a tool runs.

Self-check questions

1. **Is Azure Cloud Shell another command language like Azure CLI, or an environment for running tools?**
   Answer: It is an environment. It offers Bash or PowerShell and can run tools such as Azure CLI or Azure PowerShell.

2. **What is one distinction between Azure CLI and Azure PowerShell?**
   Answer: CLI uses `az` commands and commonly produces JSON output; Azure PowerShell uses Az cmdlets and structured PowerShell/.NET objects. Both can manage Azure resources.

3. **Does Azure Arc move an on-premises server into an Azure datacenter?**
   Answer: No. Arc can project supported non-Azure resources into Azure Resource Manager for selected management capabilities; the workload remains where it is hosted.

4. **How do ARM templates and Bicep relate?**
   Answer: Both are declarative IaC approaches for Azure Resource Manager. ARM templates use JSON; Bicep provides a more concise authoring language that is converted to an ARM template for deployment.

5. **What should you check before running a deployment command?**
   Answer: Confirm the tenant, subscription, scope, permissions, template contents, and proposed changes. Use an appropriate preview such as what-if when available, and avoid unreviewed deployments to production.

Sources

- What is Azure Resource Manager?
- What is Azure Cloud Shell?
- What is the Azure CLI?
- What is Azure PowerShell?
- Azure Arc overview
- Infrastructure as code with Azure Resource Manager templates
- What is Bicep?
- ARM template what-if operation

3.4 Monitoring and advice

Learning intent

Identify which Azure service helps answer a health, reliability, cost, performance, or application-diagnostics question. Distinguish Azure Advisor's recommendations from Azure Service Health's notices about Azure service issues and planned maintenance, Azure Monitor's telemetry-and-alerting platform, and Application Insights' application performance monitoring capabilities. Learn to choose signals and tools based on whether you are planning, investigating, or responding.

Core concepts

Azure Advisor (`mg-advisor`). Azure Advisor analyzes resource configuration and usage telemetry to provide recommendations intended to help optimize Azure deployments. Recommendations are organized into areas such as cost, reliability, performance, security, and operational excellence. A recommendation identifies an opportunity and may suggest an action; Advisor does not automatically mean the action is safe for every workload or that implementing it is mandatory. Review context, dependencies, impact, and

organizational requirements before applying a recommendation. Advisor is useful for proactive improvement—for example, spotting a cost-saving opportunity or a configuration that may affect reliability.

Advisor is not the same as monitoring an active incident. Its recommendations are generally optimization guidance based on available analysis. It does not replace resource telemetry, service incident notices, or an organization's operational review. Recommendations may take time to reflect a change after a team acts, and supported resource types vary.

Azure Service Health (`mg-service-health`). Azure Service Health gives personalized information about Azure service issues, planned maintenance, and health advisories that may affect the Azure services and regions an organization uses. It is intended to help Azure customers understand relevant events and configure alerts for issues or changes that matter to their subscriptions. This is different from a general global status page: a global status view is broad, while Service Health is tailored to the services, regions, and subscriptions relevant to the customer.

Microsoft's health experiences include related but distinct components. **Azure Status** presents global Azure service status. **Service Health** provides a personalized view of service-impacting communications and planned maintenance. **Resource Health** focuses on the health of an individual Azure resource, such as a particular virtual machine. Together, these views help answer different levels of “Is Azure healthy?” and “Is my resource affected?” questions.

Azure Monitor (`mg-azure-monitor`). Azure Monitor collects, analyzes, and acts on cloud and hybrid telemetry. Metrics are numerical measurements useful for trends and alerts; logs provide records and context; traces follow activity across components; events capture significant occurrences. Available signals depend on the source.

Azure Monitor includes tools to visualize and investigate collected data and to create alerts that notify people or trigger configured actions when conditions are met. An alert rule evaluates a signal or query and can create an alert instance when its condition is satisfied. Notification or action behavior depends on the configured action group and permissions. Monitoring is most effective when the team chooses useful signals, thresholds, scope, and response procedures rather than collecting data without a question to answer.

Log Analytics is an Azure Monitor experience for querying log and trace data in Log Analytics workspaces with Kusto Query Language (KQL). Azure Monitor also uses Azure Monitor workspaces for Prometheus and OpenTelemetry metrics, analyzed using Prometheus Query Language (PromQL). These are distinct workspace types and data stores. Do not assume that all telemetry uses the same workspace or query language. Azure Monitor provides the broad monitoring platform; Log Analytics is one component used to work with log data.

Application Insights (`mg-app-insights`). Azure Monitor Application Insights is an application performance monitoring feature of Azure Monitor. It helps developers and operators collect and analyze telemetry from applications so they can investigate availability, failures, performance bottlenecks, and behavior. Application Insights experiences include views such as application maps, live metrics, search, availability, failures, and performance. For supported scenarios, applications can use OpenTelemetry (OTel), a vendor-neutral observability framework, to instrument code and send telemetry for analysis. Setup depends on the language, hosting model, and supported instrumentation approach.

Application Insights is application-focused, while Azure Monitor covers a broader range of infrastructure, platform, and application telemetry. They are not competing choices: Application Insights is a capability in the Azure Monitor ecosystem. Monitoring data may contain sensitive information; instrumentation and data collection should follow applicable privacy, security, retention, and access requirements.

How it works, distinctions, limits, and defaults

Use the following guide to choose a tool. Some products overlap in the portal, but each starts from a different operational question.

Service or view	Main focus	Example question	Typical use
Azure Advisor	Best-practice recommendations for optimization	What changes might improve cost, reliability, performance, security, or operations?	Review recommendations and decide which actions fit.
Azure Status	Broad public view of Azure service status	Is there a widely reported Azure issue?	Check global status context; it may not show customer-specific impact.
Azure Service Health	Personalized Azure service issues, maintenance, and advisories	Is a service event or planned maintenance relevant to my subscriptions and regions?	Review targeted communications and configure alerts.
Resource Health	Health of a particular Azure resource	Is this specific VM or resource available, and what may have affected it?	Investigate individual resource availability.
Azure Monitor	Telemetry collection, analysis, visualization, and alerts	What does the measured behavior of my systems show, and should I be notified?	Monitor metrics, logs, traces, and events across supported sources.
Application Insights	Application performance and runtime behavior	Where are application failures or latency occurring?	Diagnose application availability, failures, transactions, and performance.

These tools complement one another. For example, correlate an Advisor resize recommendation with Azure Monitor metrics, a Service Health maintenance notice, and the Resource Health state of an affected resource.

Azure Monitor is not automatically comprehensive: teams may need to enable diagnostic settings, instrumentation, destinations, and alert rules. Alerts depend on configured criteria and action groups; missing alerts do not prove there was no issue. Data can be delayed, sampled, or incomplete, so choose a suitable time range and query.

Application Insights requires instrumentation or a supported automatic collection path and a configured resource. OTel support varies by language and scenario. Avoid sending secrets or sensitive personal data; use appropriate data minimization and access controls.

> **Verify:** Monitoring capabilities, supported resource types, health views, instrumentation recommendations, data retention and ingestion behavior, and portal terminology can change. Validate current documentation for the language, service, region, and plan being used. Azure Monitor workspaces and Log Analytics workspaces are different workspace types; confirm which one is required by the data source.

Example or worked scenario

A web application slows down. The on-call engineer checks Service Health for relevant incidents or maintenance, Resource Health for affected resources, and Azure Monitor metrics, logs, and alerts for evidence during the incident.

The developer uses Application Insights to investigate requests, failures, and dependencies, correlating telemetry with infrastructure evidence. After stabilization, the team reviews Advisor recommendations for longer-term improvements and assesses whether they fit the architecture.

Hands-on activity

Use an existing demo or lab subscription. Do not create costly resources or send sensitive application data to a monitoring service.

1. In the portal, inspect Advisor recommendations for an authorized subscription and note why each should be evaluated before action.
2. Compare personalized Service Health notices with Azure Status, then open a resource's Resource Health view if available.
3. In Azure Monitor, locate metrics, logs, and alert rules. In a demo Application Insights resource, inspect availability, failures, or performance and identify the telemetry source.
4. Write an operational question and specify the signal, time range, view, and alert that could answer it.

Exam or practical application

Match the noun in the requirement to the right tool. **Advice or optimization recommendations** points to Azure Advisor. **Azure outages, planned maintenance, or service-impacting notices personalized to a subscription** points to Azure Service Health. **A single resource's health** points to Resource Health. **Metrics, logs, alerts, and broad telemetry analysis** points to Azure Monitor. **Application requests, performance, availability tests, failures, and dependency diagnostics** points to Application Insights.

A common practical mistake is to use a recommendation service as an incident dashboard or to expect service-health notices to reveal application code-level latency. Start with the source that matches the question, then correlate sources. For example, combine a Service Health event with Monitor metrics and Application Insights request traces to understand both platform context and the user's application impact.

Self-check questions

1. **Which Azure service provides recommendations for cost, reliability, performance, security, and operational excellence?**
   Answer: Azure Advisor. Review its recommendations and decide whether they are appropriate before applying them.

2. **Which view is tailored to Azure service issues and maintenance that may affect the services and regions a subscription uses?**
   Answer: Azure Service Health. Azure Status is the broad public view; Resource Health focuses on an individual resource.

3. **What is the difference between Azure Monitor and Application Insights?**
   Answer: Azure Monitor is the broader telemetry and observability service. Application Insights is an application performance monitoring feature within Azure Monitor.

4. **Which data is typically queried with KQL in a Log Analytics workspace?**
   Answer: Log and trace data collected in that workspace. Azure Monitor also has a distinct workspace type for Prometheus and OpenTelemetry metrics, which uses PromQL.

5. **Does the absence of an Azure Monitor alert prove that an application is healthy?**
   Answer: No. An alert only fires when configured conditions and data sources detect its criteria. Missing instrumentation, incomplete signals, unsuitable thresholds, or configuration issues can all leave problems unalerted.

Sources

- Describe monitoring tools in Azure — AZ-900 Learn module
- Introduction to Azure Advisor
- What is Azure Service Health?
- Azure Status overview
- Resource Health overview
- Azure Monitor overview
- Log Analytics overview
- Application Insights OpenTelemetry observability overview
- Azure Monitor alerts overview